

Unidad 3 – Análisis descriptivo y visualización de datos

Introducción a la Analítica de Datos

En este notebook trabajaremos de forma práctica los conceptos de la Unidad 3:

- Analítica descriptiva y su importancia.
- Medidas básicas (frecuencia, tendencia central, dispersión, posición).
- Ejecución del proceso descriptivo.
- Herramientas de visualización de datos y su uso.
- Analítica descriptiva y cultura de datos en la organización.

El objetivo es entender los conceptos usando un conjunto de datos y responder preguntas de reflexión que se refleje la teoría con la práctica.

2. Carga de librerías y del DataSet

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from IPython.display import display, Markdown
from matplotlib.collections import PolyCollection

%matplotlib inline

# Estilo general
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.dpi"] = 125
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
plt.rcParams["axes.titleweight"] = "normal"
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 10

file_path = Path("Online-Store-Orders.xlsx")
data_url = (
    "https://raw.githubusercontent.com/SHRAJALJHARIYA/"
    "Excel-Practice-Datasets/main/ExcelDataset/Online-Store-Orders.xlsx"
)

if file_path.exists():
    orders = pd.read_excel(file_path)
else:
    orders = pd.read_excel(data_url)

orders["Date"] = pd.to_datetime(orders["Date"], errors="coerce")
```

```
print(f"Filas: {orders.shape[0]} | Columnas: {orders.shape[1]}")
orders.head()
```

Filas: 1200 | Columnas: 14

Out[1]:

	OrderID	Date	CustomerID	Product	Quantity	UnitPrice	ShippingAddress	Pay
0	ORD200000	2023-01-04	C72649	Monitor	5	570.62	928 Main St	
1	ORD200001	2024-08-23	C75739	Phone	2	151.35	823 Main St	
2	ORD200002	2024-02-27	C81728	Tablet	5	550.68	512 Main St	
3	ORD200003	2023-10-15	C33540	Chair	1	273.19	275 Main St	
4	ORD200004	2025-05-08	C81840	Printer	4	626.01	668 Main St	

Preguntas de reflexión

1. Observa las columnas del dataset

Menciona, en tus palabras, qué representa cada una de estas variables:

- Date
- Product
- Quantity
- UnitPrice
- OrderStatus
- ReferralSource
- TotalPrice

2. Desde la perspectiva de la analítica descriptiva, ¿qué tipo de preguntas "sobre el pasado" podríamos responder con estos datos?

Escribe al menos 3 preguntas (por ejemplo: "¿Cuánto hemos vendido por tipo de producto en el último año?").

Respuestas

1. Significado de las variables

- Date : fecha en la que se registró el pedido.
- Product : producto asociado a la orden.
- Quantity : número de unidades compradas.
- UnitPrice : precio de una unidad.
- OrderStatus : estado del pedido, por ejemplo entregado, enviado, pendiente, cancelado o devuelto.
- ReferralSource : canal por el que llegó o fue referido el cliente.
- TotalPrice : valor total registrado para la orden.

2. Desde la analítica descriptiva, que resume datos históricos para comprender qué ocurrió, estos datos permiten responder, entre otras, las siguientes preguntas:

- ¿Qué productos generaron mayor valor total de pedidos?
- ¿Cuál fue el valor promedio y la distribución de los pedidos?
- ¿Qué estados de pedido fueron más frecuentes?
- ¿Qué canal de referencia aportó mayor valor de pedidos?
- ¿Cómo evolucionó el valor total de los pedidos a través del tiempo?

Estas preguntas describen el pasado; no prueban por sí solas por qué ocurrió un comportamiento ni predicen necesariamente lo que ocurrirá después.

```
In [2]: # Perfil básico de las variables utilizadas en la actividad
columnas_interes = [
    "Date", "Product", "Quantity", "UnitPrice",
    "OrderStatus", "ReferralSource", "TotalPrice"
]

perfil_variables = pd.DataFrame({
    "tipo": orders[columnas_interes].dtypes.astype(str),
    "faltantes": orders[columnas_interes].isna().sum(),
    "valores_unicos": orders[columnas_interes].nunique(dropna=True)
})

perfil_variables
```

```
Out[2]:
```

		tipo	faltantes	valores_unicos
Date	datetime64[us]		0	671
Product	str		0	7
Quantity	int64		0	5
UnitPrice	float64		0	1193
OrderStatus	str		0	5
ReferralSource	str		0	5
TotalPrice	float64		0	1195

3. Vista general de los datos (analítica descriptiva básica)

```
In [3]: orders.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  --
 0   OrderID               1200 non-null   str    
 1   Date                  1200 non-null   datetime64[us]
 2   CustomerID            1200 non-null   str    
 3   Product               1200 non-null   str    
 4   Quantity              1200 non-null   int64  
 5   UnitPrice             1200 non-null   float64
 6   ShippingAddress       1200 non-null   str    
 7   PaymentMethod         1200 non-null   str    
 8   OrderStatus           1200 non-null   str    
 9   TrackingNumber        1200 non-null   str    
10   ItemsInCart           1200 non-null   int64  
11   CouponCode            891 non-null    str    
12   ReferralSource        1200 non-null   str    
13   TotalPrice            1200 non-null   float64
dtypes: datetime64[us](1), float64(2), int64(2), str(9)
memory usage: 131.4 KB

```

In [4]: `orders.describe()`

Out[4]:

	Date	Quantity	UnitPrice	ItemsInCart	TotalPrice
count	1200	1200.000000	1200.000000	1200.000000	1200.000000
mean	2024-03-22 16:58:48	2.945833	356.412750	5.485000	1053.968300
min	2023-01-01 00:00:00	1.000000	11.390000	1.000000	11.390000
25%	2023-08-03 18:00:00	2.000000	186.062500	4.000000	410.520000
50%	2024-03-23 00:00:00	3.000000	364.210000	5.000000	823.615000
75%	2024-11-08 12:00:00	4.000000	521.570000	7.000000	1578.475000
max	2025-06-30 00:00:00	5.000000	699.930000	10.000000	3456.400000
std	NaN	1.407557	197.177146	2.281983	819.856558

La analítica descriptiva, , crea un resumen de los datos históricos para apoyar análisis posteriores.

Actividad

1. Mira el resultado de `describe()` :

- ¿Qué significan la media (`mean`), el mínimo (`min`) y el máximo (`max`) de `TotalPrice`?
- ¿Qué te dice el conteo (`count`) sobre cuántos pedidos hay?

2. Explica con tus palabras una conclusión sencilla que podrías presentar a un gerente:

- Por ejemplo: "El valor promedio de los pedidos es de aproximadamente X (moneda), con un mínimo de Y y un máximo de Z".

3. ¿Por qué es útil este resumen numérico antes de hacer gráficos?

```
In [5]: # Resumen puntual para responder la actividad
resumen_totalprice = orders["TotalPrice"].agg(
    ["count", "mean", "min", "median", "max", "std", "sum"]
)
resumen_totalprice.to_frame("TotalPrice")
```

```
Out[5]:
```

	TotalPrice
count	1.200000e+03
mean	1.053968e+03
min	1.139000e+01
median	8.236150e+02
max	3.456400e+03
std	8.198566e+02
sum	1.264762e+06

```
In [6]: # Mensaje gerencial construido con los valores reales del dataset
r = resumen_totalprice
print(
    f"Se analizaron {int(r['count'])} pedidos con TotalPrice disponible. "
    f"El valor promedio fue {r['mean']:.2f}, "
    f"con un mínimo de {r['min']:.2f} y un máximo de {r['max']:.2f}. "
    f"La mediana fue {r['median']:.2f}."
)
```

Se analizaron 1200 pedidos con TotalPrice disponible. El valor promedio fue 1,053.97, con un mínimo de 11.39 y un máximo de 3,456.40. La mediana fue 823.62.

4. Medidas de frecuencia: productos y estado de las órdenes

```
In [7]: orders["Product"].value_counts()
```

```
Out[7]: Product
Printer      181
Tablet       179
Chair        178
Laptop       173
Desk         170
Monitor      163
Phone        156
Name: count, dtype: int64
```

```
In [8]: orders["OrderStatus"].value_counts()
```

```
Out[8]: OrderStatus
Cancelled    250
Returned     247
Pending      237
Shipped      235
Delivered    231
Name: count, dtype: int64
```

Preguntas de análisis

1. ¿Qué producto aparece con mayor frecuencia en las órdenes? ¿Qué interpretación de negocio se te ocurre?

```
In [9]: frecuencia_producto = orders["Product"].value_counts()
producto_frecuente = frecuencia_producto.idxmax()
n_producto_frecuente = int(frecuencia_producto.max())

display(Markdown(
    f"Respuesta: {producto_frecuente} es el producto que más aparece, con "
    f"{n_producto_frecuente} órdenes. Esto indica que es un producto muy recurrente "
    "aunque para saber cuál genera más dinero debo revisar el valor total de los "
))
```

Respuesta: Printer es el producto que más aparece, con 181 órdenes. Esto indica que es un producto muy recurrente, aunque para saber cuál genera más dinero debo revisar el valor total de los pedidos.

2. ¿Qué estados de orden aparecen? ¿Cuál es el más frecuente? ¿Qué preocupación surge si hay muchos pedidos cancelados o devueltos?

```
In [10]: frecuencia_estado = orders["OrderStatus"].value_counts()
estado_frecuente = frecuencia_estado.idxmax()
n_estado_frecuente = int(frecuencia_estado.max())
estados = ", ".join(frecuencia_estado.index.astype(str))

display(Markdown(
    f"Respuesta: aparecen {estados}. El estado más frecuente es {estado_frecuente} "
    f"con {n_estado_frecuente} pedidos. Si aumentan mucho `Cancelled` o `Returned` "
    "revisaría el proceso de compra, la entrega y la satisfacción del cliente."
))
```

Respuesta: aparecen Cancelled, Returned, Pending, Shipped, Delivered. El estado más frecuente es Cancelled, con 250 pedidos. Si aumentan mucho `Cancelled` o `Returned`, revisaría el proceso de compra, la entrega y la satisfacción del cliente.

3. Si trabajaras en servicio al cliente, ¿qué estado revisarías con más detalle y por qué?

Respuesta: revisaría primero `Returned`, porque una devolución significa que el pedido llegó más lejos en el proceso y después fue regresado. Eso puede generar costos adicionales de logística y también puede mostrar que el producto o la experiencia no cumplió con lo esperado.

```
In [11]: # Frecuencias y porcentajes
tabla_productos = pd.DataFrame({
    "frecuencia": orders["Product"].value_counts(),
    "porcentaje": (orders["Product"].value_counts(normalize=True) * 100).round(2)
})

tabla_status = pd.DataFrame({
    "frecuencia": orders["OrderStatus"].value_counts(),
    "porcentaje": (orders["OrderStatus"].value_counts(normalize=True) * 100).round(2)
})
```

```

    "porcentaje": (orders["OrderStatus"].value_counts(normalize=True) * 100).round(2)
})

display(Markdown("#### Productos"))
display(tabla_productos)

display(Markdown("#### Estados de la orden"))
display(tabla_status)

```

Productos

	frecuencia	porcentaje
Product		
Printer	181	15.08
Tablet	179	14.92
Chair	178	14.83
Laptop	173	14.42
Desk	170	14.17
Monitor	163	13.58
Phone	156	13.00

Estados de la orden

	frecuencia	porcentaje
OrderStatus		
Cancelled	250	20.83
Returned	247	20.58
Pending	237	19.75
Shipped	235	19.58
Delivered	231	19.25

```

In [13]: producto_top = orders["Product"].value_counts().idxmax()
n_producto_top = orders["Product"].value_counts().max()

status_top = orders["OrderStatus"].value_counts().idxmax()
n_status_top = orders["OrderStatus"].value_counts().max()

print(f"Producto más frecuente: {producto_top} ({n_producto_top} órdenes)")
print(f"Estado más frecuente: {status_top} ({n_status_top} órdenes)")

```

Producto más frecuente: Printer (181 órdenes)
 Estado más frecuente: Cancelled (250 órdenes)

5. Análisis de ventas: montos y cantidades

```
In [14]: # Verificar una consistencia simple: TotalPrice vs Quantity * UnitPrice
orders[["Quantity", "UnitPrice", "TotalPrice"]].head()
```

```
Out[14]:
```

	Quantity	UnitPrice	TotalPrice
0	5	570.62	2853.10
1	2	151.35	302.70
2	5	550.68	2753.40
3	1	273.19	273.19
4	4	626.01	2504.04

```
In [15]: orders[["Quantity", "UnitPrice", "TotalPrice"]].describe()
```

```
Out[15]:
```

	Quantity	UnitPrice	TotalPrice
count	1200.000000	1200.000000	1200.000000
mean	2.945833	356.412750	1053.968300
std	1.407557	197.177146	819.856558
min	1.000000	11.390000	11.390000
25%	2.000000	186.062500	410.520000
50%	3.000000	364.210000	823.615000
75%	4.000000	521.570000	1578.475000
max	5.000000	699.930000	3456.400000

Actividad

1. Calcula el promedio, la suma total y la desviación estándar de `TotalPrice`.

```
In [16]: media_total = orders["TotalPrice"].mean()
suma_total = orders["TotalPrice"].sum()
std_total = orders["TotalPrice"].std()

display(Markdown(
    f"Respuesta: el promedio es {media_total:,.2f}, la suma total es {suma_total:,.2f} y la desviación estándar es {std_total:,.2f}."
))
```

Respuesta: el promedio es 1,053.97, la suma total es 1,264,761.96 y la desviación estándar es 819.86.

2. ¿El valor de los pedidos es muy variable o relativamente estable? ¿Qué significa para la empresa tener pedidos muy diferentes en valor?

```
In [17]: cv_total = std_total / media_total * 100
display(Markdown(
    f"Respuesta: el coeficiente de variación es {cv_total:,.1f}%."
))
```



```
"Esto muestra qué tan grande es la variación comparada con el promedio. "
"Para la empresa significa que un solo valor promedio puede ocultar compras
))
```

Respuesta: el coeficiente de variación es 77.8%. Esto muestra qué tan grande es la variación comparada con el promedio. Para la empresa significa que un solo valor promedio puede ocultar compras muy pequeñas y otras mucho más altas.

3. ¿Qué te gustaría saber sobre el comportamiento de compra de estos clientes?

Respuesta: me gustaría comparar el valor de compra por producto, canal de referencia, forma de pago y fecha. También revisaría si hay clientes que vuelven a comprar, porque eso permitiría diferenciar una compra ocasional de un cliente recurrente.

In [18]: *# Ejemplo de código guía (opcional):*

```
mean_total = orders["TotalPrice"].mean()
sum_total = orders["TotalPrice"].sum()
std_total = orders["TotalPrice"].std()

mean_total, sum_total, std_total
```

Out[18]: (np.float64(1053.9683), np.float64(1264761.96), np.float64(819.8565583646455))

In [19]: `coef_variacion = std_total / mean_total * 100`

```
pd.Series({
    "Promedio TotalPrice": mean_total,
    "Suma TotalPrice": sum_total,
    "Desviación estándar": std_total,
    "Coeficiente de variación (%)": coef_variacion
}).to_frame("valor").round(2)
```

Out[19]:

	valor
Promedio TotalPrice	1053.97
Suma TotalPrice	1264761.96
Desviación estándar	819.86
Coeficiente de variación (%)	77.79

6. Histogramas: distribución del valor de los pedidos

In [20]: `fig, ax = plt.subplots(figsize=(10.5, 5.8))`

```
sns.histplot(
    data=orders,
    x="TotalPrice",
    bins=28,
    kde=True,
    alpha=0.62,
```

```

    edgecolor="white",
    linewidth=0.8,
    ax=ax
)

sns.rugplot(
    data=orders,
    x="TotalPrice",
    height=0.035,
    alpha=0.22,
    ax=ax
)

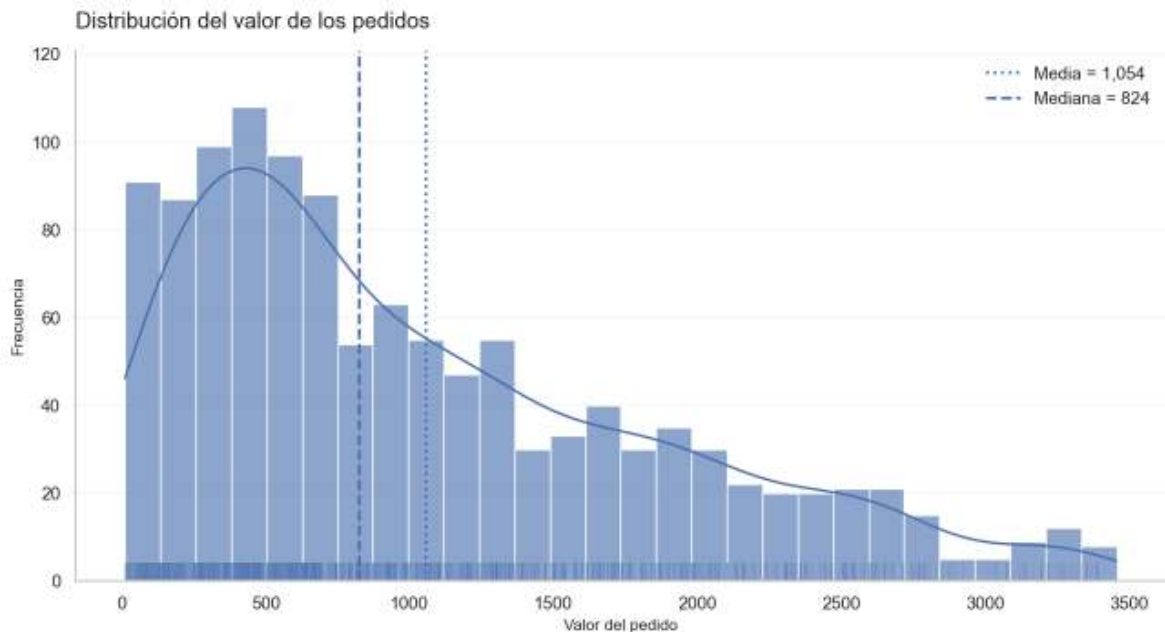
media_tp = orders["TotalPrice"].mean()
mediana_tp = orders["TotalPrice"].median()

ax.axvline(media_tp, ls=":", lw=1.8, label=f"Media = {media_tp:,.0f}")
ax.axvline(mediana_tp, ls="--", lw=1.8, label=f"Mediana = {mediana_tp:,.0f}")

ax.set_title("Distribución del valor de los pedidos", loc="left", pad=14)
ax.set_xlabel("Valor del pedido")
ax.set_ylabel("Frecuencia")
ax.legend(frameon=False)
ax.grid(axis="y", alpha=0.18)
ax.grid(axis="x", visible=False)

plt.tight_layout()
plt.show()

```



Preguntas de reflexión

1. ¿La mayoría de los pedidos son de valor bajo, medio o alto? ¿Hay pedidos extraordinariamente altos?

```

In [21]: q1 = orders["TotalPrice"].quantile(0.25)
         mediana = orders["TotalPrice"].median()
         q3 = orders["TotalPrice"].quantile(0.75)
         iqr = q3 - q1
         limite_sup = q3 + 1.5 * iqr

```

```
n_altos = int((orders["TotalPrice"] > limite_sup).sum())

display(Markdown(
    f"Respuesta: la mitad central de los pedidos está entre {q1:,.2f} y {q3:,.2f}"
    f"con una mediana de {mediana:,.2f}. Con la regla de 1,5 veces el rango inte"
    f"hay {n_altos} pedidos por encima de {limite_sup:,.2f}, que son altos respe"
))
```

Respuesta: la mitad central de los pedidos está entre 410.52 y 1,578.47, con una mediana de 823.62. Con la regla de 1,5 veces el rango intercuartílico hay 8 pedidos por encima de 3,330.41, que son altos respecto al resto.

2. Si fueras responsable de promociones, ¿a qué rango de valores de pedido te enfocarías?

Respuesta: me enfocaría en los pedidos que están cerca de la zona central de la distribución. Ahí hay más clientes y una promoción podría buscar que una compra media pase a un valor un poco mayor, en lugar de concentrarse únicamente en los pedidos más extremos.

3. ¿Cómo ayuda el histograma a detectar patrones?

Respuesta: permite ver de inmediato dónde se concentran los pedidos, si la distribución está sesgada y si hay una cola de pedidos muy altos. A partir de eso puedo preguntar si esos pedidos altos pertenecen a un producto, un canal o una forma de pago específica.

```
In [22]: # Cuartiles y detección descriptiva de valores atípicos
q1 = orders["TotalPrice"].quantile(0.25)
mediana = orders["TotalPrice"].median()
q3 = orders["TotalPrice"].quantile(0.75)
iqr = q3 - q1

limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr

atipicos = orders[
    (orders["TotalPrice"] < limite_inferior) |
    (orders["TotalPrice"] > limite_superior)
]

pd.Series({
    "Q1": q1,
    "Mediana": mediana,
    "Q3": q3,
    "IQR": iqr,
    "Límite inferior": limite_inferior,
    "Límite superior": limite_superior,
    "Número de valores atípicos": len(atipicos)
}).to_frame("valor").round(2)
```

Out[22]:

	valor
Q1	410.52
Mediana	823.62
Q3	1578.48
IQR	1167.96
Límite inferior	-1341.41
Límite superior	3330.41
Número de valores atípicos	8.00

Visualización complementaria

Además del histograma, quiero comparar cómo se distribuye el valor de los pedidos entre los distintos estados. Para eso uso un violín semitransparente y burbujas con cada observación. Así puedo ver al mismo tiempo la forma de la distribución y los datos individuales.

```
In [23]: datos_violin = orders.dropna(subset=["OrderStatus", "TotalPrice"]).copy()

orden_estados = (
    datos_violin.groupby("OrderStatus")["TotalPrice"]
    .median()
    .sort_values()
    .index
)

paleta = sns.color_palette("Set2", n_colors=len(orden_estados))

fig, ax = plt.subplots(figsize=(11, 6.5))

sns.violinplot(
    data=datos_violin,
    x="OrderStatus",
    y="TotalPrice",
    order=orden_estados,
    palette=paleta,
    inner=None,
    cut=0,
    linewidth=1.1,
    saturation=0.85,
    width=0.78,
    ax=ax
)

# Transparencia real en los cuerpos del violín
for coll in ax.collections:
    if isinstance(coll, PolyCollection):
        coll.set_alpha(0.28)

# Burbujas: observaciones individuales
muestra = datos_violin.sample(
    n=min(550, len(datos_violin)),
```

```

    random_state=42
)

sns.stripplot(
    data=muestra,
    x="OrderStatus",
    y="TotalPrice",
    order=orden_estados,
    palette=paleta,
    jitter=0.18,
    size=5.4,
    alpha=0.62,
    edgecolor="white",
    linewidth=0.55,
    ax=ax
)

# Caja muy discreta en el centro
sns.boxplot(
    data=datos_violin,
    x="OrderStatus",
    y="TotalPrice",
    order=orden_estados,
    width=0.10,
    showfliers=False,
    boxprops={"facecolor": "white", "alpha": 0.72, "zorder": 5},
    whiskerprops={"linewidth": 1.0},
    capprops={"linewidth": 1.0},
    medianprops={"linewidth": 1.7},
    ax=ax
)

medias = datos_violin.groupby("OrderStatus")["TotalPrice"].mean().reindex(orden_
conteos = datos_violin["OrderStatus"].value_counts().reindex(orden_estados)

y0, y1 = ax.get_ylim()
yr = y1 - y0

for i, estado in enumerate(orden_estados):
    media = medias.loc[estado]
    n = int(conteos.loc[estado])

    ax.scatter(
        i, media,
        s=78,
        marker="D",
        color="firebrick",
        edgecolor="white",
        linewidth=0.8,
        zorder=7
    )

    ax.annotate(
        f"{media:,.0f}",
        (i, media),
        xytext=(8, 7),
        textcoords="offset points",
        fontsize=8.5,
        bbox=dict(boxstyle="round,pad=0.22", fc="white", ec="none", alpha=0.88)
    )

```

```
ax.text(
    i,
    y0 - 0.075 * yr,
    f"n = {n}",
    ha="center",
    va="top",
    fontsize=9
)
```

```
ax.set_title("Valor de los pedidos según el estado de la orden", loc="left", pad=10)
ax.set_xlabel("")
ax.set_ylabel("Valor del pedido")
ax.grid(axis="y", alpha=0.16)
ax.grid(axis="x", visible=False)
ax.tick_params(axis="x", rotation=0)

plt.subplots_adjust(bottom=0.18, top=0.90, left=0.10, right=0.98)
plt.show()
```

C:\Users\dsalinas\AppData\Local\Temp\ipykernel_26592\1089416289.py:14: FutureWarning:

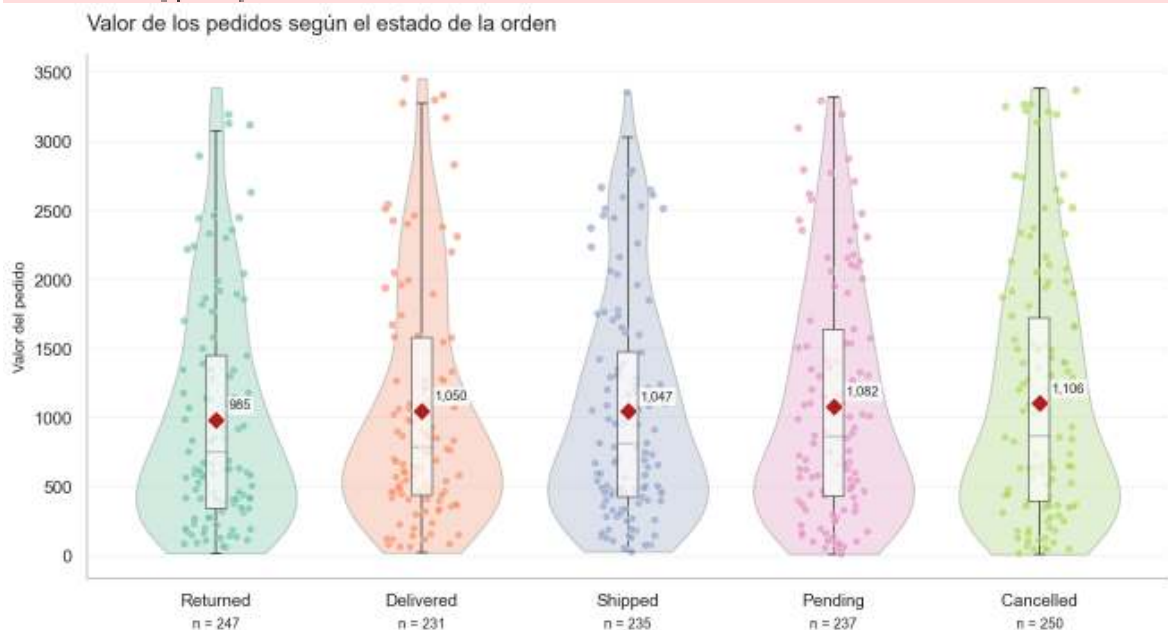
Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.violinplot(
```

C:\Users\dsalinas\AppData\Local\Temp\ipykernel_26592\1089416289.py:39: FutureWarning:

Passing 'palette' without assigning 'hue' is deprecated and will be removed in v0.14.0. Assign the 'x' variable to 'hue' and set 'legend=False' for the same effect.

```
sns.stripplot(
```



7. Gráfico de barras: ventas por tipo de producto

```
In [ ]: ventas_por_producto = orders.groupby("Product")["TotalPrice"].sum().sort_values(
ventas_por_producto
```

```
In [24]: ventas_por_producto = (
    orders.groupby("Product")["TotalPrice"]
        .sum()
        .sort_values()
)

n = len(ventas_por_producto)
colores = sns.color_palette("crest", n_colors=max(n, 3))[:n]

fig, ax = plt.subplots(figsize=(10, max(5.8, n * 0.48)))

y = np.arange(n)
valores = ventas_por_producto.values

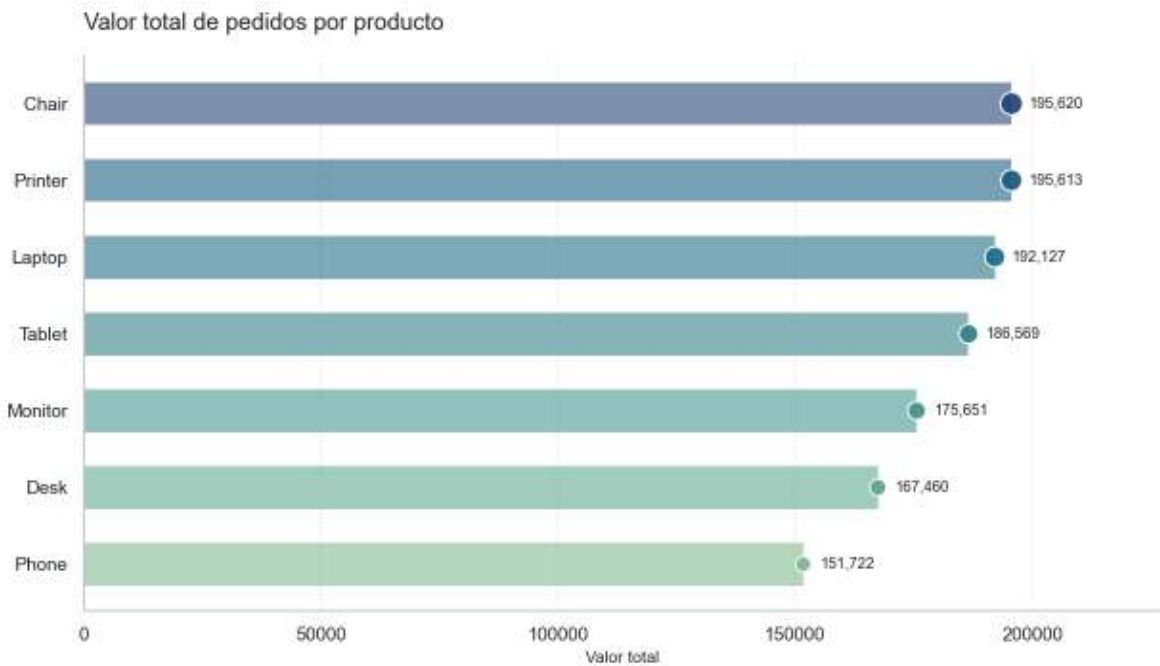
# Barras con transparencia
ax.barh(
    y,
    valores,
    color=colores,
    alpha=0.58,
    height=0.56,
    edgecolor="none"
)

# Burbujas al final de cada barra
ax.scatter(
    valores,
    y,
    s=np.linspace(85, 180, n),
    c=colores,
    alpha=0.95,
    edgecolor="white",
    linewidth=1.1,
    zorder=4
)

# Etiquetas de valor
margen = valores.max() * 0.018
for yi, valor in zip(y, valores):
    ax.text(
        valor + margen,
        yi,
        f"{valor:,.0f}",
        va="center",
        fontsize=9
    )

ax.set_yticks(y)
ax.set_yticklabels(ventas_por_producto.index)
ax.set_title("Valor total de pedidos por producto", loc="left", pad=16)
ax.set_xlabel("Valor total")
ax.set_ylabel("")
ax.grid(axis="x", alpha=0.16)
ax.grid(axis="y", visible=False)
ax.set_xlim(0, valores.max() * 1.16)
```

```
plt.tight_layout()
plt.show()
```



Preguntas de análisis

1. ¿Qué producto genera mayor valor total de pedidos? ¿Coincide con el producto más frecuente?

```
In [25]: ventas_producto = orders.groupby("Product")["TotalPrice"].sum().sort_values(ascending=False)
frecuencia_producto = orders["Product"].value_counts()

producto_valor = ventas_producto.idxmax()
valor_producto = ventas_producto.max()
producto_frecuente = frecuencia_producto.idxmax()
n_frecuente = int(frecuencia_producto.max())

coincide = "Sí" if producto_valor == producto_frecuente else "No"

display(Markdown(
    f"Respuesta: {producto_valor} es el producto con mayor valor total, con {valor_producto} valor total."
    f"El producto más frecuente es {producto_frecuente}, con {n_frecuente} órdenes."
    f"{coincide} coincide el producto más frecuente con el que más valor genera."
))
```

Respuesta: Chair es el producto con mayor valor total, con 195,620.11. El producto más frecuente es Printer, con 181 órdenes. No coincide el producto más frecuente con el que más valor genera.

2. Si fueras parte del equipo de marketing, ¿qué productos priorizarías? ¿Hay alguno que merezca revisión?

```
In [26]: producto_menor = ventas_producto.idxmin()
valor_menor = ventas_producto.min()
```



```
display(Markdown(
    f"Respuesta: priorizaría {producto_valor} porque es el que más valor aporta.
    f"También revisaría {producto_menor}, que acumula {valor_menor:,.2f}, "
    "para entender si vende menos por demanda, visibilidad, precio o disponibili
    ))
```

Respuesta: priorizaría Chair porque es el que más valor aporta. También revisaría Phone, que acumula 151,722.39, para entender si vende menos por demanda, visibilidad, precio o disponibilidad.

3. ¿Cómo ayuda este gráfico a decidir dónde hacer mayor o menor merchandising?

Respuesta: al ordenar los productos por valor total puedo ver cuáles sostienen una parte importante de los pedidos y cuáles tienen una participación menor. Eso ayuda a decidir dónde conviene reforzar promoción o revisar la estrategia comercial.

```
In [ ]: # Comparación explícita: frecuencia vs valor total
frecuencia_prod = orders["Product"].value_counts()
valor_prod = orders.groupby("Product")["TotalPrice"].sum()

producto_frecuente = frecuencia_prod.idxmax()
producto_valor = valor_prod.idxmax()

print(f"Producto con más órdenes: {producto_frecuente}")
print(f"Producto con mayor valor total: {producto_valor}")
print("¿Coinciden?:", producto_frecuente == producto_valor)
```

8. Estado de las órdenes vs valor: tabla y gráfico

```
In [27]: resumen_status = orders.groupby("OrderStatus")["TotalPrice"].agg(["count", "mean", "sum"])
resumen_status
```

```
Out[27]:
```

	count	mean	sum
OrderStatus			
Cancelled	250	1105.58	276396.21
Delivered	231	1050.22	242600.32
Pending	237	1081.55	256328.15
Returned	247	984.93	243277.70
Shipped	235	1047.49	246159.58

```
In [28]: resumen_status = orders.groupby("OrderStatus")["TotalPrice"].agg(["count", "mean", "sum"])
resumen_status_ordenado = resumen_status.sort_values("sum")

n = len(resumen_status_ordenado)
colores = sns.color_palette("mako", n_colors=max(n, 3))[:n]

fig, ax = plt.subplots(figsize=(9.5, 5.8))

y = np.arange(n)
valores = resumen_status_ordenado["sum"].values
```

```

ax.barh(
    y,
    valores,
    color=colores,
    alpha=0.55,
    height=0.55
)

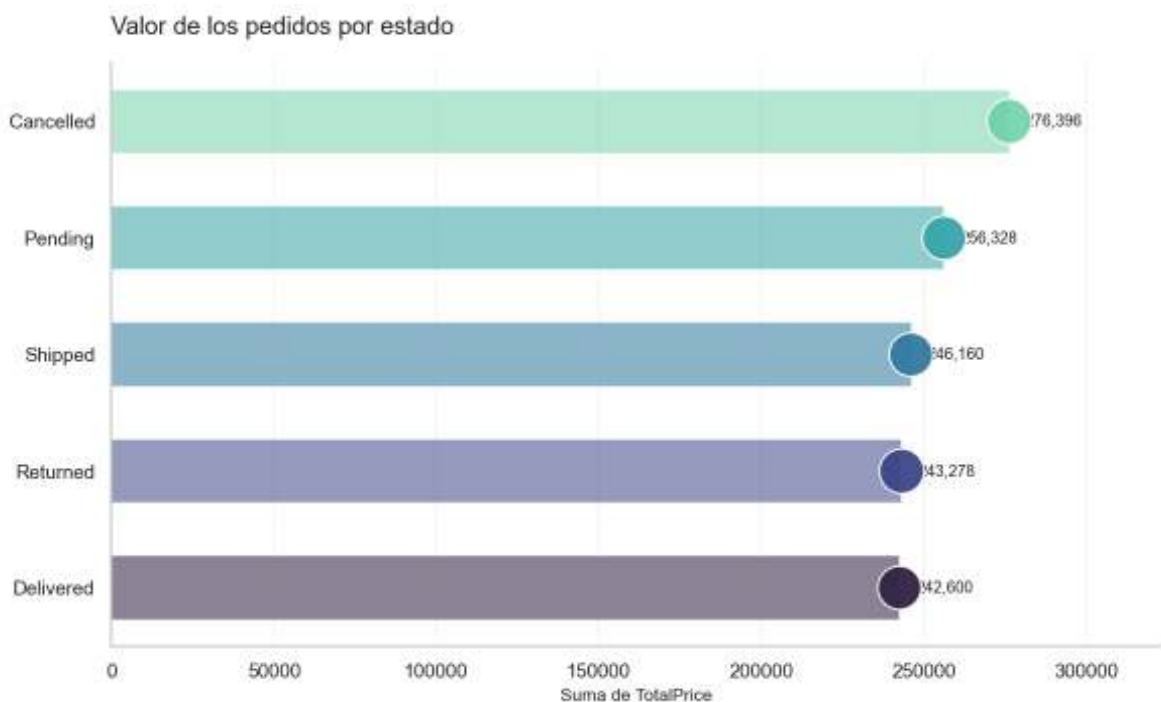
ax.scatter(
    valores,
    y,
    s=resumen_status_ordenado["count"].values * 2.4 + 55,
    c=colores,
    alpha=0.95,
    edgecolor="white",
    linewidth=1.1,
    zorder=4
)

margen = valores.max() * 0.018
for yi, valor in zip(y, valores):
    ax.text(valor + margen, yi, f"{valor:,.0f}", va="center", fontsize=9)

ax.set_yticks(y)
ax.set_yticklabels(resumen_status_ordenado.index)
ax.set_title("Valor de los pedidos por estado", loc="left", pad=16)
ax.set_xlabel("Suma de TotalPrice")
ax.set_ylabel("")
ax.grid(axis="x", alpha=0.16)
ax.grid(axis="y", visible=False)
ax.set_xlim(0, valores.max() * 1.17)

plt.tight_layout()
plt.show()

```



Preguntas de reflexión

1. ¿Qué observas sobre el valor de los pedidos en `Delivered`, `Shipped`, `Cancelled` y `Returned`?

```
In [29]: resumen_estado = orders.groupby("OrderStatus")["TotalPrice"].agg(["count", "mean")
estado_valor = resumen_estado["sum"].idxmax()
valor_estado = resumen_estado["sum"].max()

display(Markdown(
    f"Respuesta: {estado_valor} es el estado con mayor valor acumulado, con {val
    "También compararía `Cancelled` y `Returned`, pero no trataría esos valores
    ))
```

Respuesta: `Cancelled` es el estado con mayor valor acumulado, con 276,396.21. También compararía `Cancelled` y `Returned`, pero no trataría esos valores automáticamente como ventas efectivas.

2. Si hay muchos pedidos `Cancelled` o `Returned`, ¿qué preguntas te surgen?

Respuesta: revisaría qué productos aparecen más en esos casos, si las incidencias se concentran en alguna forma de pago o canal, en qué momento se cancela el pedido y si existen problemas de entrega o expectativas sobre el producto.

3. ¿Qué equipos deberían revisar este resumen y qué decisiones podrían tomar?

Respuesta: lo revisaría con logística, servicio al cliente, comercial y gerencia. Con esos datos se pueden investigar devoluciones, revisar tiempos de entrega, detectar productos problemáticos y seguir la evolución de cancelaciones.

9. Origen del tráfico: ReferralSource

```
In [30]: orders["ReferralSource"].value_counts()
```

```
Out[30]: ReferralSource
Instagram    259
Email        258
Google       241
Facebook     228
Referral     222
Name: count, dtype: int64
```

```
In [31]: ventas_por_canal = orders.groupby("ReferralSource")["TotalPrice"].sum().sort_val
ventas_por_canal
```

```
Out[31]: ReferralSource
Instagram    275285.45
Email        261888.55
Google       258441.48
Facebook     258410.98
Referral     226815.58
Name: TotalPrice, dtype: float64
```

```
In [32]: ventas_por_canal = (
    orders.groupby("ReferralSource")["TotalPrice"]
```

```

        .sum()
        .sort_values()
    )

n = len(ventas_por_canal)
colores = sns.color_palette("flare", n_colors=max(n, 3))[:n]

fig, ax = plt.subplots(figsize=(9.5, 5.8))

y = np.arange(n)
valores = ventas_por_canal.values

ax.barh(
    y,
    valores,
    color=colores,
    alpha=0.56,
    height=0.55
)

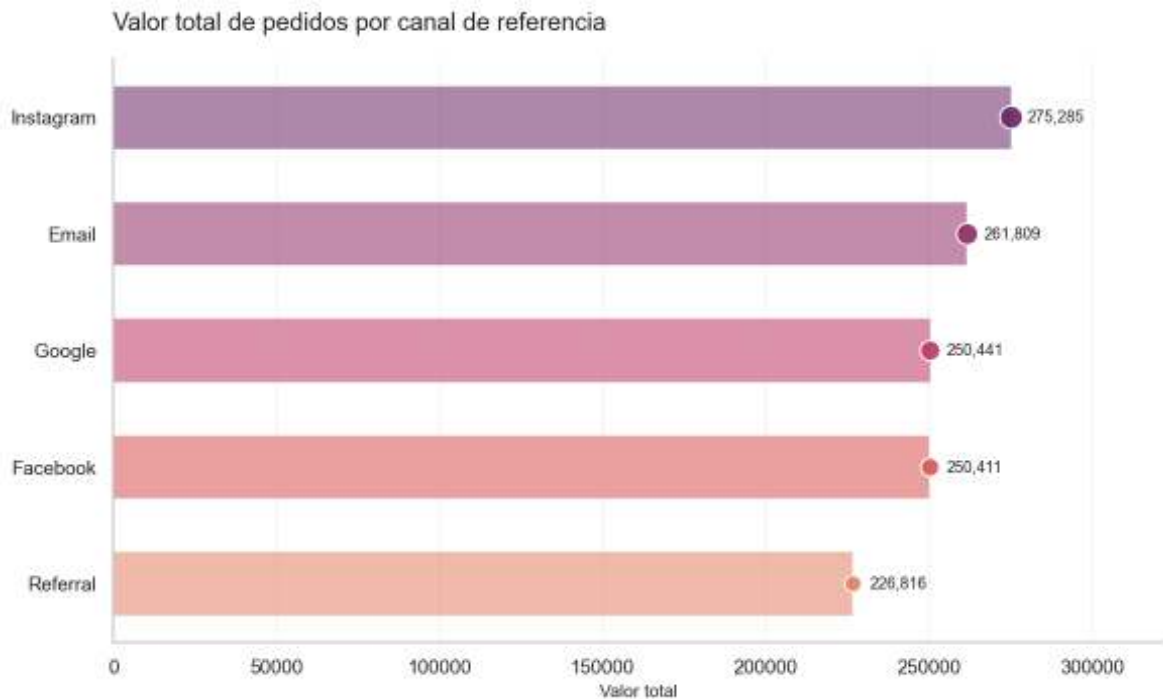
ax.scatter(
    valores,
    y,
    s=np.linspace(90, 170, n),
    c=colores,
    alpha=0.95,
    edgecolor="white",
    linewidth=1.1,
    zorder=4
)

margen = valores.max() * 0.018
for yi, valor in zip(y, valores):
    ax.text(valor + margen, yi, f"{valor:,.0f}", va="center", fontsize=9)

ax.set_yticks(y)
ax.set_yticklabels(ventas_por_canal.index)
ax.set_title("Valor total de pedidos por canal de referencia", loc="left", pad=1)
ax.set_xlabel("Valor total")
ax.set_ylabel("")
ax.grid(axis="x", alpha=0.16)
ax.grid(axis="y", visible=False)
ax.set_xlim(0, valores.max() * 1.17)

plt.tight_layout()
plt.show()

```



Preguntas de análisis

1. ¿Qué canal genera mayor valor total de pedidos?

```
In [33]: resumen_canal = orders.groupby("ReferralSource")["TotalPrice"].agg(["count", "mean"])
canal_top = resumen_canal["sum"].idxmax()
valor_top = resumen_canal["sum"].max()

display(Markdown(
    f"Respuesta: {canal_top} es el canal con mayor valor total de pedidos, con {valor_top:.2f}."
))
```

Respuesta: Instagram es el canal con mayor valor total de pedidos, con 275,285.45.

2. Si fueras responsable de publicidad digital, ¿qué canal priorizarías? ¿Un canal pequeño todavía podría tener potencial?

```
In [34]: canal_bajo = resumen_canal["sum"].idxmin()
valor_bajo = resumen_canal["sum"].min()

display(Markdown(
    f"Respuesta: empezaría revisando {canal_top}, porque es el canal que más valor aporta."
    f"También analizaría {canal_bajo}, que suma {valor_bajo:.2f}."
    "Un canal pequeño todavía puede ser útil si consigue clientes a menor costo o atrae clientes que vuelven a comprar."
))
```

Respuesta: empezaría revisando Instagram, porque es el canal que más valor aporta.

También analizaría Referral, que suma 226,815.58. Un canal pequeño todavía puede ser útil si consigue clientes a menor costo o atrae clientes que vuelven a comprar.

3. ¿Cómo explicarías este gráfico a una persona que no se siente cómoda con los números?

Respuesta: cada barra representa cuánto valor de pedidos llegó por un canal. La barra más larga identifica el canal que más aportó. Eso no significa necesariamente que sea el más rentable, porque aquí no se muestran los costos de publicidad.

```
In [35]: resumen_canal = orders.groupby("ReferralSource")["TotalPrice"].agg(
        ["count", "mean", "sum"]
    ).sort_values("sum", ascending=False)

resumen_canal["porcentaje_valor"] = (
    resumen_canal["sum"] / resumen_canal["sum"].sum() * 100
)

canal_top = resumen_canal["sum"].idxmax()

display(resumen_canal.round(2))
print(f"Canal con mayor valor total de pedidos: {canal_top}")
```

	count	mean	sum	porcentaje_valor
ReferralSource				
Instagram	259	1062.88	275285.45	21.77
Email	250	1047.23	261808.55	20.70
Google	241	1039.18	250441.48	19.80
Facebook	228	1098.29	250410.90	19.80
Referral	222	1021.69	226815.58	17.93

Canal con mayor valor total de pedidos: Instagram

10. Cultura de datos y flujo de datos, reflexión conceptual

Analítica descriptiva y cultura de datos en la tienda en línea

Con base en los análisis que hiciste:

Preguntas de reflexión

1. PERSONAS:

- ¿Qué perfiles dentro de la tienda (no solo de tecnología) deberían aprender a leer estos dashboards?
- ¿Cómo les explicarías la importancia de revisar regularmente estas visualizaciones?

2. PROCESOS:

- Diseña un proceso sencillo (en pasos) para que cada mes se:
 - A. extraigan los datos,

- B. actualicen los gráficos,
- C. se discutan en una reunión de equipo.

3. TECNOLOGÍA:

- Menciona al menos dos herramientas (por ejemplo, Excel, Power BI, Python, etc.) con las que se podrían construir dashboards similares a los de este notebook.
- ¿Qué ventajas ofrece una herramienta de visualización para "apoyar la toma de decisiones acertadas" y "ahorrar tiempo" en comparación con ver solo tablas de datos?

4. DATOS:

- ¿Qué problemas podrían surgir si los datos de esta tienda están incompletos, tienen errores o no se actualizan a tiempo?
- Relaciónalo con la idea de perfilar la calidad de los datos y resolver problemas desde su origen.

1. Personas

Gerencia, comercial, marketing, logística, inventarios y servicio al cliente deberían aprender a leer estos tableros. No necesitan saber programar; lo importante es que puedan reconocer cambios en ventas, productos, canales y estados de las órdenes para tomar decisiones con información real.

2. Procesos

Cada mes haría este proceso:

1. Extraer las órdenes del sistema.
2. Revisar datos faltantes, duplicados y errores.
3. Actualizar los cálculos y las gráficas.
4. Comparar los resultados con el mes anterior.
5. Reunir al equipo para discutir qué cambió y qué acciones se van a tomar.
6. Guardar las decisiones para comprobar después si funcionaron.

3. Tecnología

Usaría Python para limpiar, analizar y automatizar los cálculos, y Power BI para construir tableros que otras personas puedan consultar fácilmente. Excel también sirve para revisiones rápidas. La principal ventaja de una visualización es que permite detectar diferencias y tendencias mucho más rápido que leyendo cientos de filas.

4. Datos

Si los datos están incompletos, tienen errores o llegan tarde, las conclusiones también pueden estar equivocadas. Por eso primero revisaría la calidad de los datos y corregiría el

problema desde donde se genera, antes de construir indicadores o gráficos.

```
In [36]: # Perfil de calidad básico del dataset
calidad = pd.DataFrame({
    "tipo": orders.dtypes.astype(str),
    "faltantes": orders.isna().sum(),
    "porcentaje_faltantes": (orders.isna().mean() * 100).round(2),
    "valores_unicos": orders.nunique(dropna=True)
})

print("Filas duplicadas completas:", orders.duplicated().sum())
calidad
```

Filas duplicadas completas: 0

```
Out[36]:
```

	tipo	faltantes	porcentaje_faltantes	valores_unicos
OrderID	str	0	0.00	1200
Date	datetime64[us]	0	0.00	671
CustomerID	str	0	0.00	1189
Product	str	0	0.00	7
Quantity	int64	0	0.00	5
UnitPrice	float64	0	0.00	1193
ShippingAddress	str	0	0.00	655
PaymentMethod	str	0	0.00	5
OrderStatus	str	0	0.00	5
TrackingNumber	str	0	0.00	1200
ItemsInCart	int64	0	0.00	10
CouponCode	str	309	25.75	3
ReferralSource	str	0	0.00	5
TotalPrice	float64	0	0.00	1195

11. Cierre del notebook (Opcional)

Ejercicio final – Elegir el gráfico según lo que quiero mostrar

Historia 1 – Comparar productos

Quiero mostrar qué productos generan mayor valor total de pedidos.

¿Qué gráfico usarías? ¿Qué variable iría en cada eje?

Respuesta: usaría barras horizontales. En el eje X pondría el valor total de los pedidos y en el eje Y los productos. Como estoy comparando categorías, las barras permiten

ordenar los productos y reconocer rápidamente cuáles aportan más.

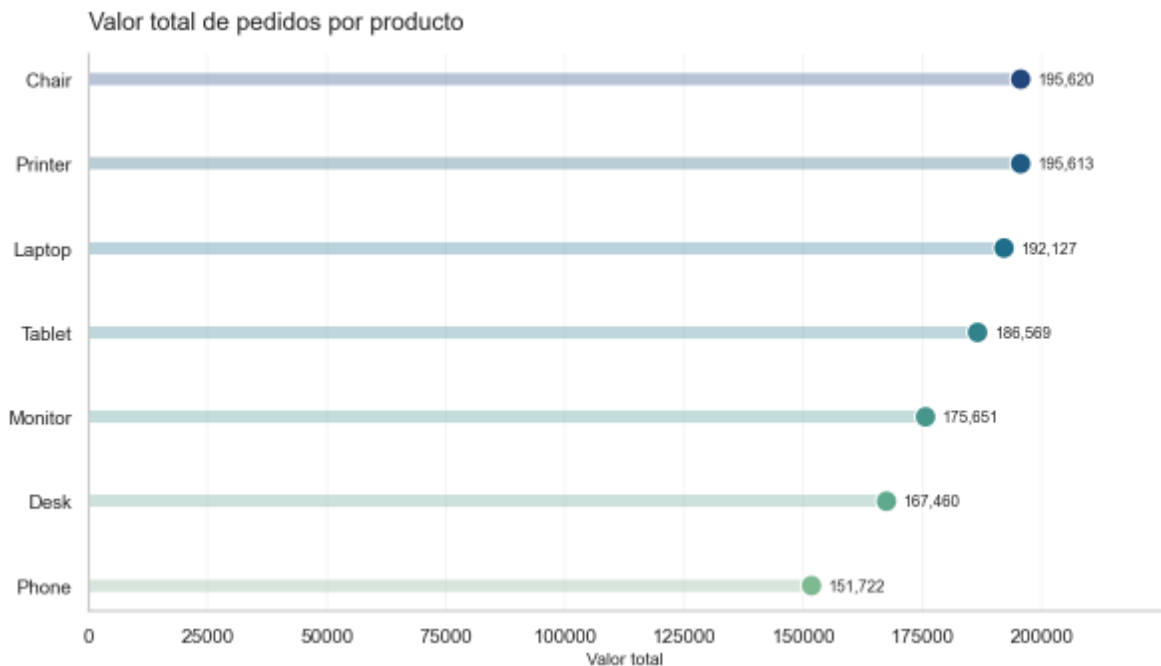
```
In [37]: hist1 = orders.groupby("Product")["TotalPrice"].sum().sort_values()

fig, ax = plt.subplots(figsize=(9.6, 5.6))
colors = sns.color_palette("crest", n_colors=len(hist1))
y = np.arange(len(hist1))

ax.hlines(y, 0, hist1.values, color=colors, linewidth=7, alpha=0.30)
ax.scatter(hist1.values, y, s=155, c=colors, edgecolor="white", linewidth=1.2, zorder=2)

for yi, v in zip(y, hist1.values):
    ax.text(v + hist1.max()*0.018, yi, f"{v:,.0f}", va="center", fontsize=9)

ax.set_yticks(y)
ax.set_yticklabels(hist1.index)
ax.set_xlim(0, hist1.max()*1.15)
ax.set_title("Valor total de pedidos por producto", loc="left", pad=14)
ax.set_xlabel("Valor total")
ax.set_ylabel("")
ax.grid(axis="x", alpha=0.16)
ax.grid(axis="y", visible=False)
plt.tight_layout()
plt.show()
```



Historia 2 – Distribución del valor de los pedidos

Quiero saber si la mayoría de los pedidos son bajos, medios o altos y si hay valores extremos.

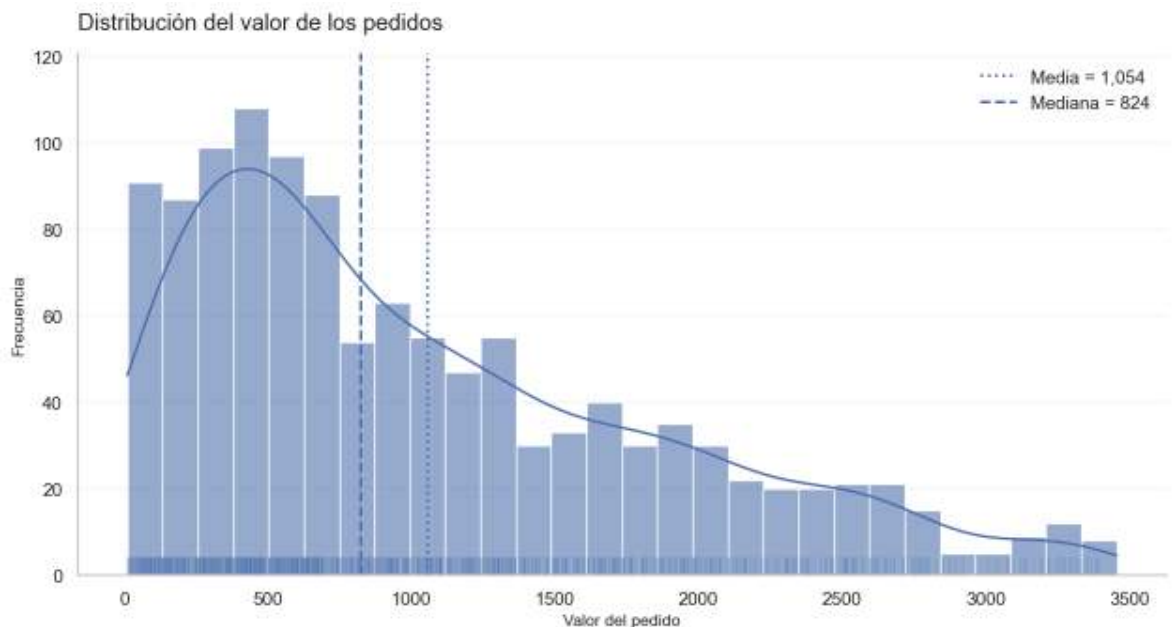
¿Qué gráfico usarías?

Respuesta: usaría un histograma como gráfico principal porque muestra cuántos pedidos caen en cada rango. Como complemento usaría un violín con burbujas, porque permite ver la forma de la distribución y los pedidos individuales.

```
In [38]: fig, ax = plt.subplots(figsize=(10.2, 5.6))
sns.histplot(data=orders, x="TotalPrice", bins=28, kde=True,
             alpha=0.58, edgecolor="white", linewidth=0.8, ax=ax)
sns.rugplot(data=orders, x="TotalPrice", height=0.035, alpha=0.18, ax=ax)

ax.axvline(orders["TotalPrice"].mean(), ls=":", lw=1.7,
          label=f"Media = {orders['TotalPrice'].mean():.0f}")
ax.axvline(orders["TotalPrice"].median(), ls="--", lw=1.7,
          label=f"Mediana = {orders['TotalPrice'].median():.0f}")

ax.set_title("Distribución del valor de los pedidos", loc="left", pad=14)
ax.set_xlabel("Valor del pedido")
ax.set_ylabel("Frecuencia")
ax.legend(frameon=False)
ax.grid(axis="y", alpha=0.16)
ax.grid(axis="x", visible=False)
plt.tight_layout()
plt.show()
```



```
In [39]: datos_total = orders["TotalPrice"].dropna().to_frame()

fig, ax = plt.subplots(figsize=(10.2, 4.2))

sns.violinplot(
    data=datos_total,
    x="TotalPrice",
    inner=None,
    cut=0,
    linewidth=1.1,
    color=sns.color_palette("Set2")[0],
    ax=ax
)

# Hacer el violín transparente para que las burbujas sean visibles
for coll in ax.collections:
    if isinstance(coll, PolyCollection):
        coll.set_alpha(0.25)

muestra_total = datos_total.sample(n=min(500, len(datos_total)), random_state=42)
```

```

sns.stripplot(
    data=muestra_total,
    x="TotalPrice",
    jitter=0.115,
    size=5.0,
    alpha=0.58,
    color=sns.color_palette("Set2")[0],
    edgecolor="white",
    linewidth=0.55,
    ax=ax
)

# Mediana y media como referencias discretas
mediana_total = datos_total["TotalPrice"].median()
media_total = datos_total["TotalPrice"].mean()

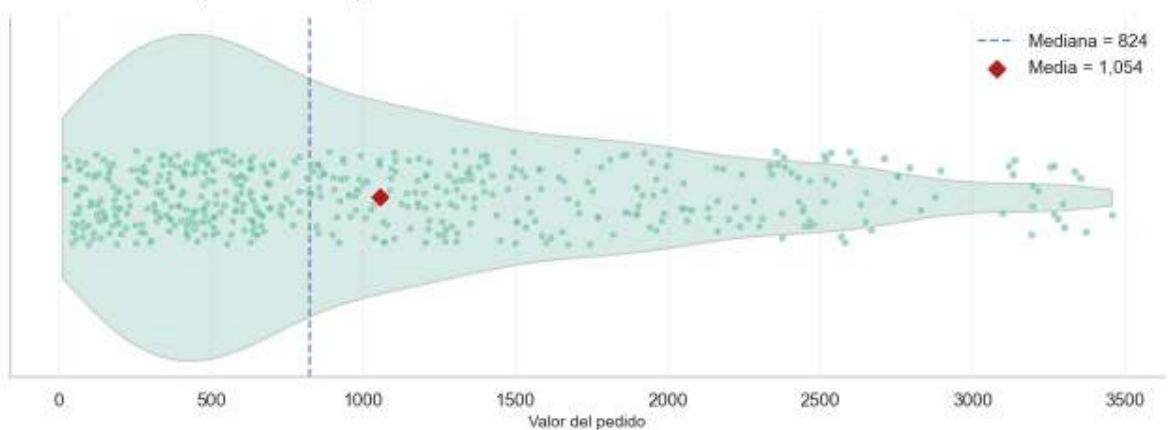
ax.axvline(mediana_total, ls="--", lw=1.4, alpha=0.8, label=f"Mediana = {mediana_total}")
ax.scatter(media_total, 0, s=85, marker="D", color="firebrick",
           edgecolor="white", linewidth=0.8, zorder=5, label=f"Media = {media_total}")

ax.set_title("Distribución de los pedidos: violín y observaciones individuales",
ax.set_xlabel("Valor del pedido")
ax.set_ylabel("")
ax.set_yticks([])
ax.grid(axis="x", alpha=0.15)
ax.grid(axis="y", visible=False)
ax.legend(frameon=False, loc="upper right")

plt.tight_layout()
plt.show()

```

Distribución de los pedidos: violín y observaciones individuales



Historia 3 – Seguimiento en el tiempo

Quiero mostrar cómo cambia el valor total de los pedidos por mes.

¿Qué gráfico usarías y qué patrón buscarías?

Respuesta: usaría una línea. En el eje X pondría los meses y en el eje Y el valor total de los pedidos. Buscaría aumentos, caídas, picos y periodos que se repitan.

```

In [40]: ventas_mes = (
    orders.dropna(subset=["Date"])
    .set_index("Date")

```

```

        .resample("MS")["TotalPrice"]
        .sum()
    )

fig, ax = plt.subplots(figsize=(10.4, 5.2))
ax.plot(ventas_mes.index, ventas_mes.values, linewidth=2.2, marker="o",
        markersize=4.5, alpha=0.90)
ax.fill_between(ventas_mes.index, ventas_mes.values, alpha=0.10)
ax.scatter(ventas_mes.index, ventas_mes.values, s=28, alpha=0.70)

ax.set_title("Evolución mensual del valor de los pedidos", loc="left", pad=14)
ax.set_xlabel("Mes")
ax.set_ylabel("Valor total")
ax.grid(axis="y", alpha=0.16)
ax.grid(axis="x", alpha=0.08)
plt.tight_layout()
plt.show()

```



Historia 4 – Composición por canal de referencia

Quiero mostrar qué porcentaje del valor total viene de cada canal.

¿Qué gráfico usarías y por qué?

Respuesta: usaría una dona porque aquí quiero mostrar cómo se reparte un total. El porcentaje permite comparar la participación de cada canal sin perder de vista que todos juntos representan el 100 %.

```

In [41]: canal = orders.groupby("ReferralSource")["TotalPrice"].sum().sort_values(ascendi
porcentajes = canal / canal.sum() * 100
colors = sns.color_palette("Set2", n_colors=len(canal))

fig, ax = plt.subplots(figsize=(9.2, 6.3))

wedges, _, autotexts = ax.pie(
    canal.values,
    labels=None,
    colors=colors,
    startangle=90,
    counterclock=False,

```

```

    autopct=lambda p: f"{p:.1f}%" if p >= 4 else "",
    pctdistance=0.78,
    wedgeprops={"width": 0.38, "edgecolor": "white", "linewidth": 1.4},
    textprops={"fontsize": 9}
)

# Los porcentajes quedan dentro de la dona, no "volando"
for text in autotexts:
    text.set_bbox(dict(boxstyle="round,pad=0.20", fc="white", ec="none", alpha=0.5))

# Centro Limpio
ax.text(0, 0.06, "100%", ha="center", va="center", fontsize=19)
ax.text(0, -0.11, "del valor total", ha="center", va="center", fontsize=9)

legend_labels = [
    f"{nombre} · {pct:.1f}%"
    for nombre, pct in zip(canal.index, porcentajes.values)
]

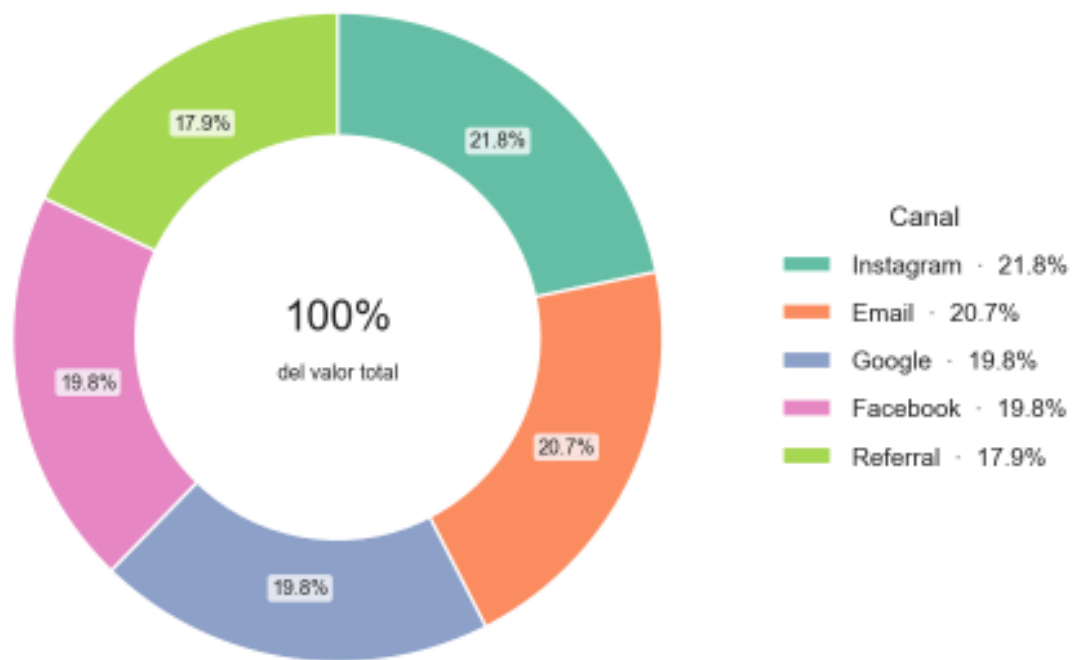
ax.legend(
    wedges,
    legend_labels,
    title="Canal",
    loc="center left",
    bbox_to_anchor=(1.02, 0.5),
    frameon=False,
    labelspring=1.0
)

ax.set_title("Participación del valor de pedidos por canal", loc="left", pad=16)
ax.set_aspect("equal")

plt.subplots_adjust(left=0.05, right=0.76, top=0.88, bottom=0.08)
plt.show()

```

Participación del valor de pedidos por canal



Historia 5 – Forma de pago y valor del pedido

Quiero comparar si el valor de los pedidos cambia según la forma de pago.

¿Qué gráfico usarías?

Respuesta: usaría violines con burbujas. El violín permite comparar la forma y la dispersión de cada grupo, mientras que las burbujas muestran los pedidos individuales y evitan que la distribución quede como una figura sólida sin datos visibles.

```
In [42]: if "PaymentMethod" in orders.columns:
    datos_pago = orders.dropna(subset=["PaymentMethod", "TotalPrice"]).copy()

    orden_pago = (
        datos_pago.groupby("PaymentMethod")["TotalPrice"]
        .median()
        .sort_values()
        .index
    )

    palette = sns.color_palette("Set2", n_colors=len(orden_pago))
    fig, ax = plt.subplots(figsize=(10.8, 6.2))

    sns.violinplot(
        data=datos_pago,
        x="PaymentMethod",
        y="TotalPrice",
        order=orden_pago,
        palette=palette,
        inner=None,
```

```

        cut=0,
        linewidth=1.1,
        saturation=0.85,
        width=0.76,
        ax=ax
    )

    # Transparencia real en los violines
    for coll in ax.collections:
        if isinstance(coll, PolyCollection):
            coll.set_alpha(0.25)

muestra_pago = datos_pago.sample(n=min(520, len(datos_pago)), random_state=42)

sns.stripplot(
    data=muestra_pago,
    x="PaymentMethod",
    y="TotalPrice",
    order=orden_pago,
    palette=palette,
    jitter=0.18,
    size=5.0,
    alpha=0.58,
    edgecolor="white",
    linewidth=0.55,
    ax=ax
)

sns.boxplot(
    data=datos_pago,
    x="PaymentMethod",
    y="TotalPrice",
    order=orden_pago,
    width=0.10,
    showfliers=False,
    boxprops={"facecolor": "white", "alpha": 0.70, "zorder": 5},
    whiskerprops={"linewidth": 1.0},
    capprops={"linewidth": 1.0},
    medianprops={"linewidth": 1.7},
    ax=ax
)

medias_pago = datos_pago.groupby("PaymentMethod")["TotalPrice"].mean().reindex(orden_pago)

for pos, metodo in enumerate(orden_pago):
    media = medias_pago.loc[metodo]
    ax.scatter(pos, media, s=70, marker="D", color="firebrick",
               edgecolor="white", linewidth=0.8, zorder=6)
    ax.annotate(
        f"{media:,.0f}",
        (pos, media),
        xytext=(7, 7),
        textcoords="offset points",
        fontsize=8.5,
        bbox=dict(boxstyle="round,pad=0.20", fc="white", ec="none", alpha=0.5)
    )

ax.set_title("Valor de los pedidos según la forma de pago", loc="left", pad=10)
ax.set_xlabel("")
ax.set_ylabel("Valor del pedido")

```

```

ax.tick_params(axis="x", rotation=0)
ax.grid(axis="y", alpha=0.16)
ax.grid(axis="x", visible=False)

plt.subplots_adjust(bottom=0.14, top=0.89, left=0.10, right=0.98)
plt.show()
else:
    print("El dataset no contiene la columna PaymentMethod.")

```

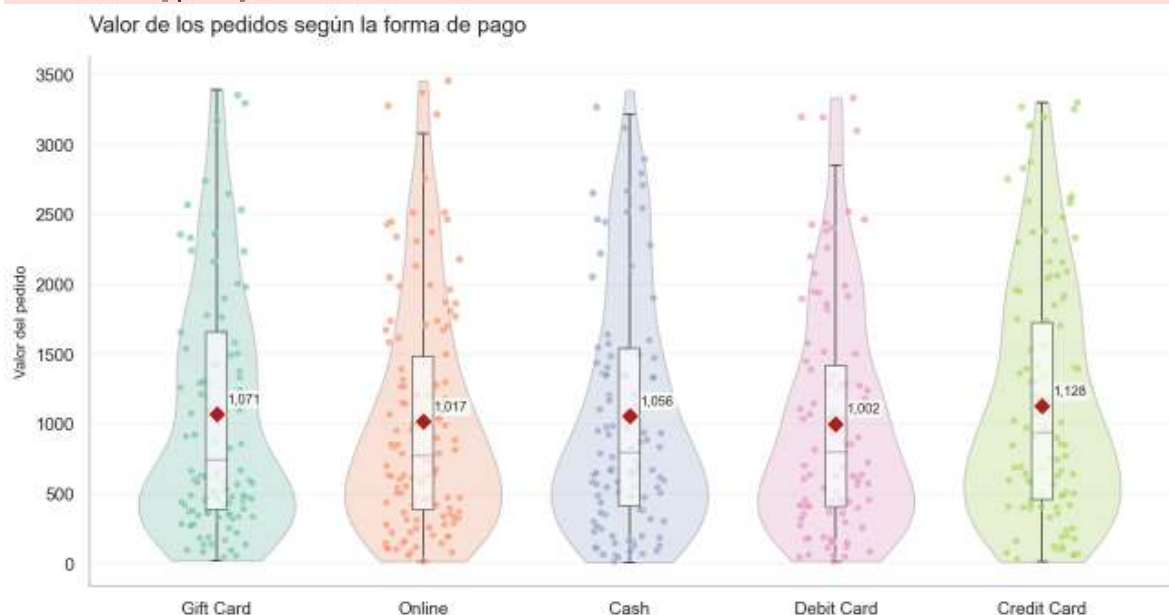
C:\Users\dsalinas\AppData\Local\Temp\ipykernel_26592\1495405991.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

C:\Users\dsalinas\AppData\Local\Temp\ipykernel_26592\1495405991.py:35: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.stripplot(
```



Historia 6 – Variabilidad del valor de los pedidos

Quiero mostrar no solo el promedio, sino también la dispersión, los cuartiles y los valores extremos.

¿Qué gráfico usarías?

Respuesta: usaría un diagrama de cajas y bigotes. La media sola resume todo en un número; la caja muestra dónde está la mitad central de los pedidos y ayuda a reconocer valores alejados del resto.

```
In [43]: fig, ax = plt.subplots(figsize=(10.0, 4.2))
```



```

sns.boxplot(
    data=orders,
    x="TotalPrice",
    width=0.28,
    showfliers=False,
    color=sns.color_palette("Set2")[2],
    boxprops={"alpha": 0.45},
    ax=ax
)

muestra_box = orders[["TotalPrice"]].dropna().sample(
    n=min(420, orders["TotalPrice"].notna().sum()),
    random_state=42
)

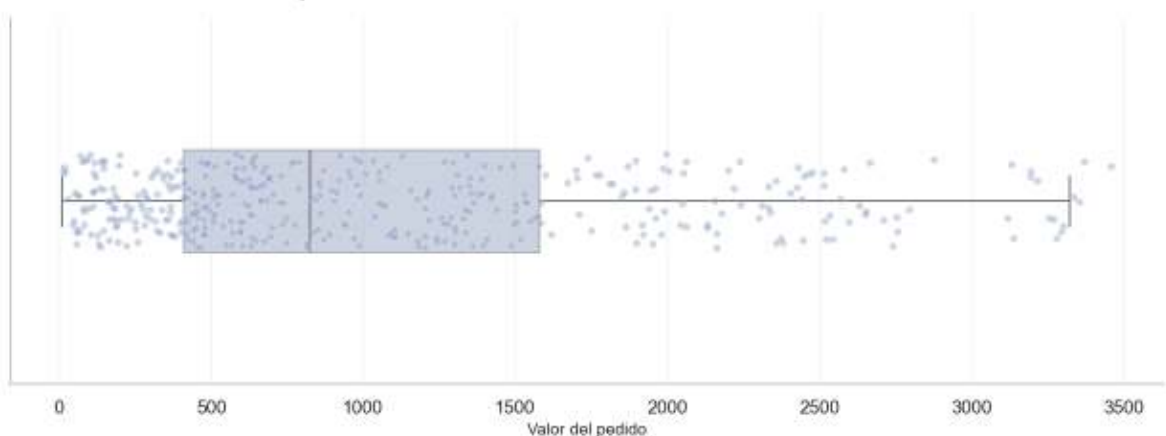
sns.stripplot(
    data=muestra_box,
    x="TotalPrice",
    jitter=0.13,
    size=4.5,
    alpha=0.42,
    color=sns.color_palette("Set2")[2],
    edgecolor="white",
    linewidth=0.45,
    ax=ax
)

ax.set_title("Variabilidad del valor de los pedidos", loc="left", pad=14)
ax.set_xlabel("Valor del pedido")
ax.set_ylabel("")
ax.set_yticks([])
ax.grid(axis="x", alpha=0.15)
ax.grid(axis="y", visible=False)

plt.tight_layout()
plt.show()

```

Variabilidad del valor de los pedidos



Reflexión final

¿Qué pregunta debería hacerme primero al elegir una gráfica?

Respuesta: primero me preguntaría "¿qué quiero mostrar?". Después escogería el gráfico.

Si empiezo por el gráfico que sé hacer, puedo terminar forzando una visualización que

no responde bien la pregunta.

Para una persona no técnica explicaría la Historia 3 así: uso una línea porque quiero ver cómo cambian los pedidos con el tiempo. Cada punto representa un mes. La forma de la línea permite reconocer rápidamente aumentos, caídas y picos.

Notebook terminado.

```
In [44]: # Revisión rápida del notebook
# Esta celda no cambia los datos; solo ayuda a comprobar que las variables principales
columnas_necesarias = [
    "Date", "Product", "Quantity", "UnitPrice",
    "OrderStatus", "ReferralSource", "TotalPrice"
]

faltantes = [c for c in columnas_necesarias if c not in orders.columns]

if faltantes:
    print("Faltan columnas:", faltantes)
else:
    print("Revisión terminada: están disponibles todas las columnas principales")
```

Revisión terminada: están disponibles todas las columnas principales usadas en el análisis.